Lasso Regression

- Lasso = Least Absolute shrinkage and Selection operator)
- method that reduces overfitting and selects important features by forcing some Co-efficients to Zero

$$\min_w \left(\frac{1}{n} \sum (y_i - x_i w)^2 + \lambda \sum |w_i| \right)$$

$\min_w$ ⇒ minimize with respect to w

- we are trying to find "optimal" weights

$$\frac{1}{n} \sum (y_i - x_i w)^2$$

- MSE
- smaller = better fit

$$\lambda \sum |w_i|$$

- Add penalty on large Co-efficients
- λ controls how strong the penalty is
  - large λ → more shrinking → more zeroes
  - small λ → less effect

<u>programming</u>

Scanned with OKEN Scanner

```
import pandas as pd
from sklearn.linear_model import lasso ;
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

- lasso → will solve optimization problem
- train_test_split → divide the dataset into training & testing
- mean_squared_error → computes

$$\frac{1}{n}\sum(y_i-\hat{y}_i)$$

```
data = pd.DataFrame({
        'hours':[1,2,3,4,5],
        'sleep':[8,7,6,5,4],
        'score':[50,55,60,65,70]
    })

x = data[['hours', 'sleep']]
y = data['score']
```

- x → 2D data y → 1D Data

x = [hours sleep] y = [score]

$$x = \begin{bmatrix} 1 & 8 \\ 2 & 7 \\ 3 & 6 \\ 4 & 5 \\ 5 & 4 \end{bmatrix} \qquad y = \begin{bmatrix} 50 \\ 55 \\ 60 \\ 65 \\ 70 \end{bmatrix}$$

Scanned with OKEN Scanner

- we define the model

$$y = wx_1 + wx_2 + b$$

where
x_1 = hours $\quad$ x_2 = sleep

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)

- we divide the data into
  - training = 100 − 20 = 80%
  - testing = 0.2 = 20%

model = lasso(alpha = 1.0)

$\alpha = \lambda$

objective becomes

$$Loss = MSE + \lambda(|w_1| + |w_2|)$$

- controls how strong weights are penalized

model.fit(x_train, y_train)

$$\min_{w_1, w_2, b} \left(\frac{1}{n}\sum\left(y_i - (w_1 x_{i1} + w_2 x_{i2} + b)\right)\right) + \lambda(|w_1| + |w_2|)$$

- minimizes error
- Applies L_1 penalty
- may force $w_2 = 0$

Scanned with OKEN Scanner

y_pred = model.predict(x_test)

- It uses the learned weights (w_1, w_2) from model.fit()

mse = mean_squared_error(y_test, y_pred)

- measures the prediction error on unseen data

$$mse = \frac{1}{n} \sum (y_i - \hat{y}_i)$$

print(model.coeff_)
print(model.intercept_)

- coff_ $\rightarrow [w_1, w_2]$
- intercept_ $\rightarrow b$

Ex:-

$y = 6(\text{hours}) + 0(\text{sleep}) + 40$

- Here Lasso removes sleep

Scanned with OKEN Scanner